

Fundamentos de la programación

Grado en Desarrollo de Videojuegos

Convocatoria ordinaria, mayo 2025

Indicaciones generales:

- Se entregarán los archivos `Program.cs`, `Tablero.cs` y `Lista.cs`, con las clases correspondientes. Se proporciona una **plantilla** con el esquema del programa y algunos métodos ya implementados.
 - En todos los archivos las líneas 1 y 2 serán comentarios de la forma:
`// Nombre Apellido1 Apellido2`
`// Laboratorio, puesto`
 - **Lee atentamente el enunciado** e implementa el programa tal como se pide, con las clases, tipos de datos, métodos, parámetros y requisitos que se especifican. No puede modificarse la representación propuesta, ni alterar los parámetros de los métodos especificados. No se especifica el modo de paso de los parámetros en los métodos (`out`, `ref`, ...), que debe determinar el alumno. Pueden implementarse todos los métodos adicionales que se consideren oportunos, especificando claramente su cometido, parámetros, etc.
 - **El programa debe compilar y funcionar correctamente, y estar bien estructurado y comentado.** Se valorarán la claridad, la concisión y la eficiencia.
 - La **entrega**: se realizará a través del servidor FTP de los laboratorios.
Importante: verificar que los archivos entregados son los requeridos.
-

Vamos a implementar el **Kakurasu**, un puzzle que se juega en un tablero $N \times N$ (con $N \leq 9$), en el que se deben marcar casillas con negro para que la suma de las posiciones coincida con los números indicados para cada fila y cada columna. Por ejemplo, para $N = 3$, podemos tener en pantalla:

	1	2	3						
1		0						4	
2		.						5	
3		.						0	
	1	2	3						

	1	2	3						
1		0						4	
2		.						5	
3		.						0	
	1	2	3						

	1	2	3						
1		0	0					4	
2		.						5	
3		.						0	
	1	2	3						

	1	2	3						
1		0	0					4	
2		.	0					5	
3		.						0	
	1	2	3						

En nuestra representación, las casillas marcadas aparecen con '0'. A la izquierda se muestra el estado inicial, sin resolver. El tablero de juego está delimitado por las líneas verticales y horizontales. La fila superior y la columna izquierda (que tienen 1,2,3) son simplemente la numeración de las filas y columnas, para facilitar al jugador la identificación de las posiciones. La columna a la derecha (4,5,0) determina la *suma objetivo* de las filas; la fila inferior (1,2,3) es la *suma objetivo* para las columnas. En este caso, las posiciones negras de la primera fila deben sumar 4, las de la segunda 5 y las de la tercera 0; las posiciones negras de las columnas deben sumar 1, 2 y 3, respectivamente.

El segundo tablero corresponde a un estado intermedio en el que se ha marcado la esquina superior izquierda (con '0'). Como la suma objetivo de esa columna es 1, no puede contener más negros y hemos anotado el resto de la columna con '.' (aunque no es necesario, el jugador

puede hacer estas anotaciones para facilitar el juego). En el tercer tablero hemos marcado la última posición de la fila 1, de modo que las posiciones marcadas en esa fila (la 1 y la 3), alcanzan la suma objetivo, 4. En el último tablero hemos marcado otras dos posiciones en la segunda fila, de modo que el juego está terminado. Nótese que no es necesario anotar con '.' las casillas restantes para comprobar la terminación del juego.

La posición del cursor se muestra poniendo el fondo de la casilla correspondiente en gris. El jugador podrá moverse por la cuadrícula con las flechas de cursor, marcar negro con 'n', anotar blanco con 'b' y borrar con la barra espaciadora.

Para desarrollar el juego implementaremos una clase `Tablero` que gestione la presentación de la cuadrícula en pantalla, el movimiento del cursor, el marcado de casillas, etc. En el archivo `Tablero.cs` se proporciona una plantilla con la representación que utilizaremos:

```
public class Tablero {
    private enum Casilla { NoDef, Negra, Blanca }; // estado de la casilla
    private int N; // lado de la matriz de juego
    private Casilla[,] mat; // matriz de juego
    private int[] objetivosFila, // sumas objetivo por fila
                objetivosColumna; // sumas objetivo por columna
    private int fil, col; // fila y columna del cursor
}
```

Para las casillas definimos el tipo enumerado `Casilla` con los 3 posibles estados de una casilla. `N` determina el lado de la matriz de juego. La cuadrícula se guarda la matriz `mat`, con valores de tipo `Casilla` y tamaño $N \times N$. Las sumas objetivo por fila y columna se guardan en los arrays `objetivosFila` y `objetivosColumna` (de tamaño `N`) y la posición del cursor en `fil` y `col`.

Para esta clase implementaremos los siguientes métodos:

- `[1] public Tablero(int[,] datos)` (constructora): recibe una matriz de enteros `datos` que contiene la matriz de juego y los objetivos por fila y columna, y construye la representación descrita arriba. A continuación se muestra un ejemplo de matriz de juego y su presentación en pantalla:

```
datos = {
    { -1, -1, -1, 4 },
    { -1, 0, -1, 5 },
    { -1, 1, -1, 0 },
    { 1, 2, 3, 0 } }
```

	1	2	3	
1	0	0		4
2	.	0	0	5
3	.			0
	1	2	3	

La matriz `datos` tiene tamaño $(N+1) \times (N+1)$, de modo que la última fila y columna contienen las sumas objetivo (el 0 en la última fila, última columna es irrelevante). Los valores -1 representan casilla no definida, 0 es casilla negra y 1 blanca (`NoDef`, `Negra`, `Blanca`). Esta constructora debe inicializar el valor `N`, la matriz `mat` y los arrays de objetivos, así como la posición del cursor en (0,0).

- `[1.5] public void Render()`: muestra la cuadrícula en pantalla en el formato descrito, **con la numeración de filas y columnas y las líneas delimitadoras**, y teniendo en cuenta que $N \leq 9$. Las casillas negras se muestran como '0', las blancas como '.' y las no definidas como '. '. Todas ellas se muestran con un espacio por delante para mejorar el aspecto visual. Para situar el cursor utilizaremos `Console.SetCursorPosition(left,top)` y para mostrarlo en fondo gris utilizaremos el método `Console.BackgroundColor = ConsoleColor.Gray`.

En el archivo `Program` se incluye una matriz `ex1` con el tablero que hemos utilizado como ejemplo. Desde el método `Main` ya se puede invocar a la constructora y al renderizado para mostrarlo en pantalla.

En `Program` se da implementado el método `LeeInput` para leer el input de usuario y codificarlo en caracteres simples. A continuación, en la clase `Tablero` implementaremos::

- **[0.5]** `void ProcesaInput(char c)`: procesa el input codificado en `c`:
 - `'u', 'd', 'l', 'r'` (up, down, left, right): actualiza la posición del cursor (`fil`, `col`) según la dirección indicada, controlando que no se salga de los límites de la cuadrícula (no hace nada si se sale).
 - `'n', 'b', 's'` (negro, blanco, limpiar casilla): coloca en `mat` el valor correspondiente, en la posición del cursor (`LeeInput` devuelve `'s'` para la barra espaciadora).

Con este método ya se puede implementar una primera versión del bucle principal (en `Program.cs`). Los siguientes métodos servirán para comprobar la terminación del juego, cuando las casillas marcadas verifiquen las condiciones requeridas. La clase `Lista` se da implementada en el archivo `Lista.cs` y se utilizará en dichos métodos.

- **[0.5]** `int SumaFila(int fil)`: calcula la suma de las posiciones marcadas de la fila `fil`.
`int SumaColumna(int col)`: calcula la suma de posiciones marcadas de la columna `col`.
- **[1]** `void Incorrectas(Lista fils, Lista cols)`: crea una lista `fils` de índices con las filas incorrectas, es decir, aquellas en las que la suma de posiciones no coincide con la suma objetivo. Análogamente, crea la lista `cols` de columnas incorrectas. Para ello se utilizan los dos métodos anteriores.
- **[0.5]** `bool Terminado()`: utilizando el método anterior devuelve `true` si todas las filas y columnas verifican las reglas del juego (suma de posiciones marcadas igual a la suma objetivo), o `false` en caso contrario. Este método se utilizará en el bucle principal del `Main` para determinar si el juego ha terminado.

Con estos métodos ya se puede implementar una primera versión del juego completando el bucle principal (en `Main`), que terminará cuando el usuario aborte el juego (con *Escape*) o cuando esté terminado. A continuación haremos algunas extensiones:

- **[0.5]** `void Pista()`: utilizando el método `Incorrectas` determina las filas y columnas incorrectas del tablero y, en caso de existir, las muestra en pantalla por debajo de tablaro. Para ello, la clase de las listas incluye un método `AString()` que devuelve el contenido de una lista en un string.
- **[1]** `string AStringRec()`: implementa una versión recursiva del método `AString()` en la clase `Lista`. **Las versiones no recursivas puntúan 0.**
- **[1.5]** `int[,] LeerArchivo(string ruta)`: se implementará en la clase `Program`. Recibe en `ruta` un nombre de archivo que contiene un tablero de juego y devuelve una matriz de enteros con dicho tablero. El archivo tendrá el siguiente formato:

```
3
-1 -1 -1 4
-1  0 -1 5
-1  1 -1 0
 1  2  3
```

El primer 3 indica el tamaño del tablero (3x3), y las siguientes líneas contienen la matriz de juego en el mismo formato descrito en la constructora de la clase (la última fila y columna contienen las sumas objetivo).

El método debe lanzar excepción si el archivo no existe o tiene un formato incorrecto (número incorrecto de filas o columnas, o contenido no esperado). Se proporcionan dos ejemplos de archivos en `ex1.txt` y `ex2.txt`.

[2] Completar el método `Main` de modo que el usuario pueda:

- Elegir arrancar el juego un con tablero de archivo (que solicitará por teclado) o bien con el del ejemplo inicial.
- Solicitar pista (con la tecla `'p'`) para ver las filas y columnas incorrectas (con el método `Pista`). Este input se procesará en el propio bucle principal (no en `ProcesaInput`).
- Abortar el juego con la tecla `'Escape'` (el método `LeeInput` lo devuelve como `'q'`).